

TARGET ARCHITECTURE PROPOSAL

CMR Continuous Data and Multi-UI Framework

A JSON-first, event-driven platform that turns continuous information into safe, observable case workflows across citizen, operator, supervisor, knowledge and integration experiences.

Prepared for proof-of-concept and pilot planning
Baseline: CMR Specialist v2.0 | Target horizon: pilot to production
September 2026

SECTION 01

Executive proposal

Evolve the existing prototype into a user-facing platform without losing its strongest feature: software-controlled, evidence-backed actions.

Recommendation: adopt a modular, event-driven architecture with a canonical JSON envelope, a shared Experience API and several role-specific interfaces. Keep MockDB behind the same connector contract that a future CMR integration must satisfy.

The architectural decision

CORE

One trusted case model

Every UI reads and writes the same case, task, approval, evidence and audit objects. There are no UI-owned copies of truth.

FLOW

Continuous data as events

Every accepted payload becomes a versioned event. Events can be replayed, traced, subscribed to and evaluated.

EXPERIENCE

Many focused interfaces

Citizens, operators, supervisors, knowledge owners and integrators see views designed for their task - not a generic administrative screen.

CONTROL

CMR operations stay sealed

All database-like activity crosses CHECK and WRITE contracts. Approval, evidence and idempotency remain enforced at execution time.

What the pilot should prove

Proof point	Observable evidence	Pilot acceptance
Continuous intake	JSON submitted by UI, webhook and batch appears as one ordered case timeline.	No lost accepted events; duplicates produce no duplicate side effects.
User-facing control	Each persona completes its top tasks without database or API knowledge.	At least 90% scripted task completion; operator usability score at least 75/100.
Safe action	Sensitive operations pause, show evidence and require a named decision.	100% approval and evidence gates; zero unauthorized writes.
Replaceable backend	The same conformance suite runs against MockDB and the future connector.	No UI or workflow rewrite when the backend adapter changes.

SECTION 02

Architecture principles

The framework should feel simple to users while remaining explicit, testable and conservative underneath.

01

JSON is the public contract

All inbound commands, events and outbound views use documented JSON schemas. Files are represented by metadata and secure object references.

02

Store before inference

Persist the accepted event and correlation identifiers before classification, extraction, retrieval or planning.

03

Commands are not direct writes

A user expresses intent through a command; policy and workflow decide whether a CHECK or WRITE may execute.

04

Read models serve the UI

Interfaces receive task-focused JSON views rather than assembling complex screens from raw relational tables.

05

Live updates are projections

WebSocket or server-sent updates notify a UI after committed state changes; they never become a second source of truth.

06

Evolution is versioned

Schemas, events, workflows and connectors carry versions, compatibility rules and deprecation windows.

Current-to-target evolution

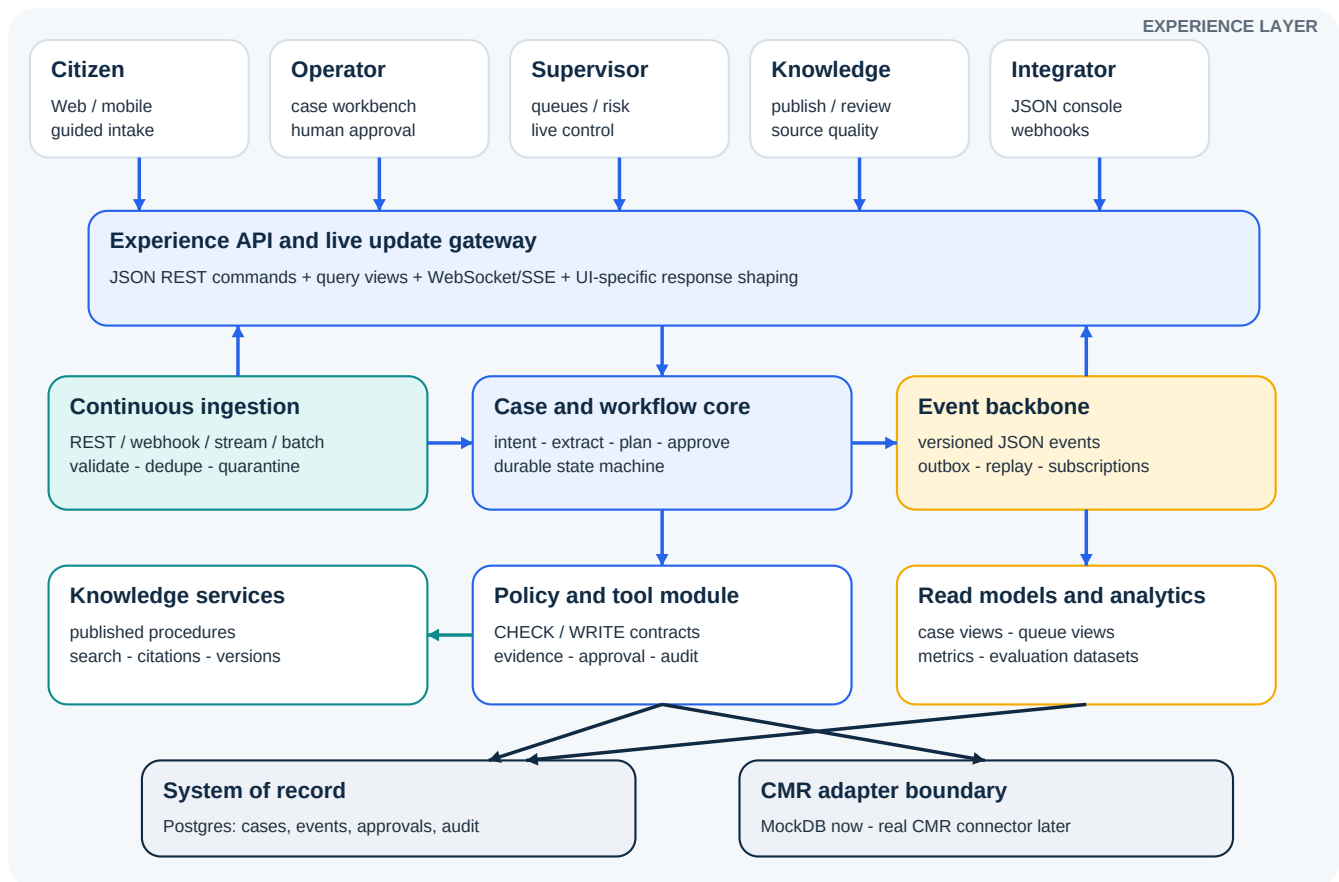
Current prototype	Target framework	Reason
Single operator console	Role-specific web/PWA interfaces on a shared design system	Less cognitive load and safer manipulation.
Synchronous HTTP intake	HTTP plus webhooks, streams, batch and replay	Continuous, resilient data feeding.
Generic response payloads	UI-specific read models returned as JSON	Stable screens and faster UI development.
Direct table polling	Committed events and live subscriptions	Immediate feedback without inconsistent state.
MockDB implementation	Versioned CMR connector contract	Replace the simulator without changing workflows.

Start as a modular monolith plus worker and Postgres. Introduce an external message broker only when measured throughput, replay volume or team ownership makes it necessary.

SECTION 03

Target architecture at a glance

Five experience surfaces share one Experience API. Continuous inputs become durable events, workflows and read models before any UI sees a result.



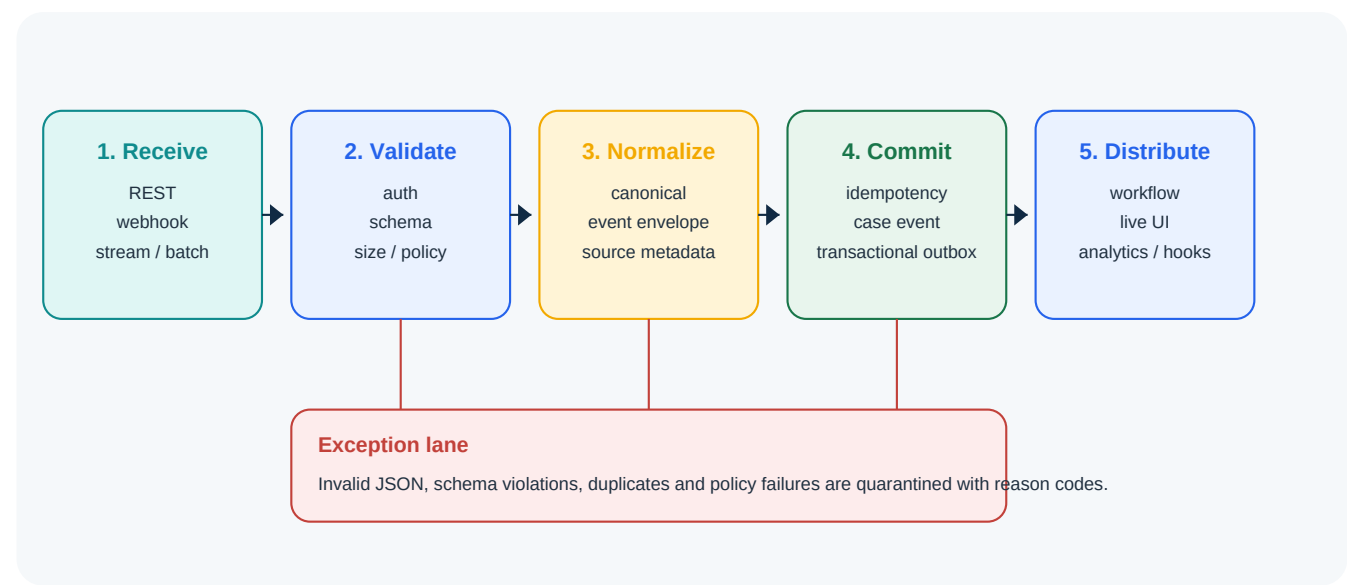
Primary boundaries

Boundary	Owns	Must not do
Experience layer	Navigation, forms, accessibility, role views, optimistic presentation	Call MockDB or CMR directly
Experience API	Authentication, JSON contracts, command/query endpoints, live subscriptions	Invent business state outside the case core
Case/workflow core	State transitions, extraction, planning, approvals and durable execution	Bypass tool policy
Tool module	Typed CHECK/WRITE execution, evidence, idempotency and audit	Interpret free-form user text
CMR adapter	Translate formulated calls to MockDB or real backend	Expose backend-specific behavior to UIs

SECTION 04

Continuous JSON ingestion

Every channel uses the same acceptance pipeline so continuous data is validated, deduplicated, traceable and safe to replay.



Supported feeds

Feed	Use	Delivery rule
REST command	Interactive UI actions	Immediate 202 Accepted with event and case identifiers
Webhook	Partner and payment notifications	Signed payload, retry-safe idempotency key
Event stream	High-frequency institutional integration	At-least-once delivery; consumer deduplication
Batch JSONL	Migration, backfill and evaluation datasets	Manifest, row-level results and resumable import
File metadata	PDF/image evidence	Object reference in JSON; malware/type/size checks before use

Ingestion guarantees

At-least-once intake The sender may retry safely. The idempotency key identifies the logical submission.	Ordered per case Sequence numbers and optimistic version checks prevent stale UI commands from silently winning.
Quarantine, do not discard Invalid payloads retain source, reason code and redacted diagnostics for correction or replay.	Backpressure by design Rate limits, bounded queues and admission rules keep continuous feeds from starving interactive users.

SECTION 05

Canonical JSON contract

A small versioned envelope carries identity, routing, provenance and payload. Domain schemas remain specific and independently versioned.

```
{
  "schema": "cmr.event/1.0",
  "event_id": "evt_01K8...",
  "event_type": "case.input.received",
  "occurred_at": "2026-09-26T10:15:30Z",
  "source": {
    "channel": "operator-web",
    "actor_id": "usr_204"
  },
  "correlation": {
    "case_id": "case_7f3...",
    "request_id": "req_91a...",
    "causation_id": null,
    "idempotency_key": "branch17-rrr-123456789012-v1"
  },
  "data": {
    "message_type": "payment_update",
    "text": "Payment made yesterday",
    "fields": {
      "remita_rrr": "123456789012"
    },
    "attachments": []
  }
}
```

Contract rules

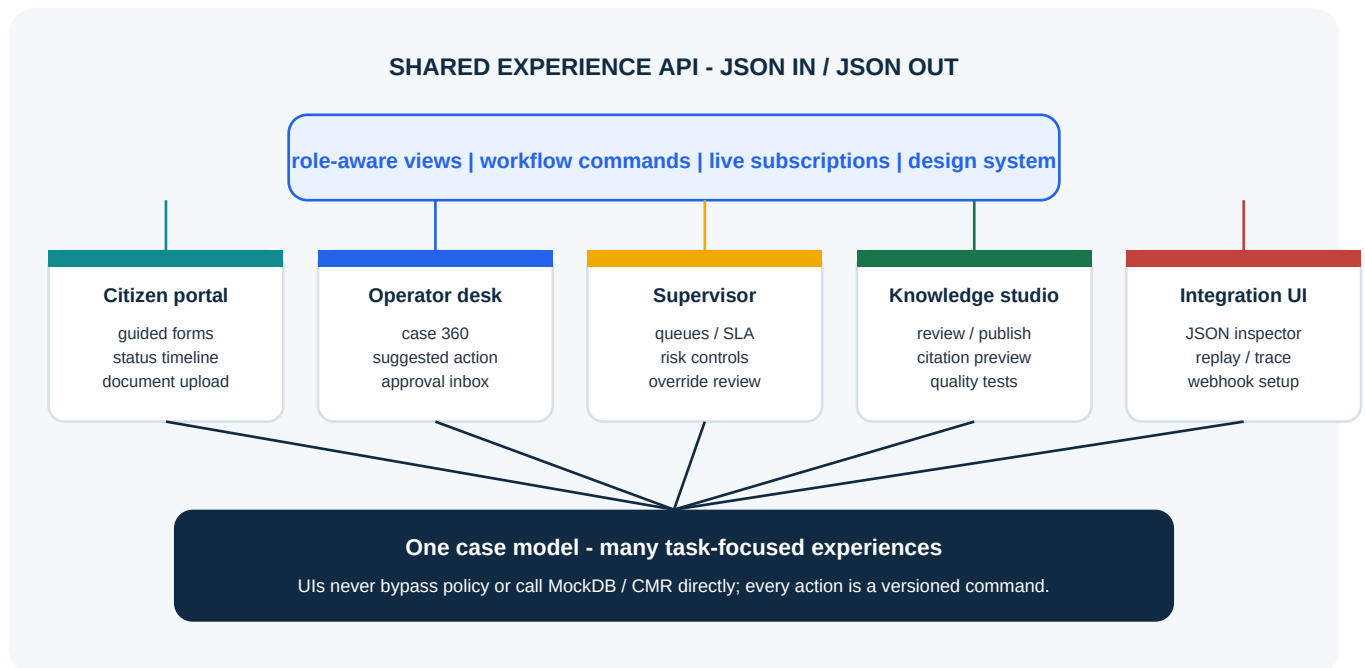
Rule	Decision
Versioning	Major versions may break compatibility; minor versions only add optional fields.
Identifiers	Opaque string IDs with prefixes; never expose sequential database keys.
Time	UTC ISO 8601 at boundaries; preserve original source time separately when supplied.
Errors	Stable machine-readable code, human message, field path, retryability and trace ID.
PII	Mark schema fields by sensitivity; redact logs and minimize UI projections by role.
Concurrency	Commands include expected_case_version; conflicts return the latest view for resolution.

Return 202 Accepted for durable asynchronous work, then publish status changes through the live update channel. A successful HTTP response means accepted - not completed.

SECTION 06

Multi-UI experience architecture

Use one design system and one Experience API, but give each persona a narrowly optimized workspace.



Interface	Primary users	Essential interactions
Citizen portal / PWA	Citizens and vehicle owners	Guided request, save/resume, evidence upload, status timeline, plain-language prompts
Operator workbench	Support and case officers	Case 360, missing-data checklist, suggested next action, approval request, conversation history
Supervisor control tower	Team leads and risk owners	Queue health, SLA risk, bulk assignment, approval review, incident and override audit
Knowledge studio	Policy and content owners	Upload, compare, validate, publish, archive and test cited answers before activation
Integration console	Technical administrators	Schema browser, webhook keys, sample payloads, delivery traces, quarantine and replay

Interaction standard

- Every destructive or sensitive action uses a review screen showing target, evidence, expected effect and approval requirement.
- Every case screen exposes a human-readable timeline derived from immutable events, with technical details available on demand.
- Every status has an explanation and a recommended next action; raw internal state names are not presented without translation.
- Accessibility target: WCAG 2.2 AA, keyboard completion of all operator workflows and mobile-first citizen forms.

SECTION 07

User-facing manipulation model

Users manipulate cases through safe commands and contextual controls, while the framework converts those intentions into governed workflow transitions.

Operator case workspace

Region	What the user sees	What the system does
Case header	Identity summary, status, owner, SLA and risk flags	Returns a role-filtered case_summary projection
Guided next step	One recommended action plus alternatives	Evaluates workflow state, missing fields and policy
Evidence panel	Receipts, citations, checks and validation state	Links immutable evidence references to claims and tool calls
Conversation	Messages and structured facts in one chronology	Stores raw input; versions extracted fields with provenance
Action drawer	Preview, reason, effect, approval and confirmation	Creates a command; never performs a direct backend write
Timeline	Plain-language case history with expandable detail	Projects case events and tool audits into a readable view

Example: payment issue

	Citizen or operator submits JSON	Payload is accepted, validated and attached to the case.
	System verifies receipt	CHECK.receipt.lookup returns evidence or a clear failure reason.
	UI shows proposed link	Account, paid status, evidence and expected change are previewed.
	Approver decides	Decision, actor, reason and timestamp are recorded.
	WRITE executes once	Idempotency prevents duplicates; result and audit appear live.

The design goal is confidence, not density: show the user what changed, why it changed, who authorized it and what happens next.

SECTION 08

Workflow and event model

Separate immutable facts from current state. Events explain what happened; projections provide fast, task-specific views.

Case state machine

State	Entry condition	User-facing meaning	Exit
open	First accepted input	We received the request.	Classify and extract
waiting_for_input	Required data is missing or invalid	We need specific information from you.	Valid follow-up event
queued	Task is executable	Your request is ready for processing.	Worker lease
running	Worker owns the lease	Checks are in progress.	Result, approval need or error
waiting_approval	Sensitive action is proposed	An authorized person must review this action.	Approval decision
completed	Validated final result stored	The request finished; evidence is available.	New related request only
failed / dead_letter	Terminal or exhausted failure	The request needs specialist attention.	Manual correction and replay

Core event families

Input events

case.created, case.input.received, attachment.accepted, field.observed

Decision events

intent.classified, plan.proposed, policy.evaluated, approval.decided

Execution events

task.queued, task.leased, tool.checked, tool.write_attempted, task.completed

Knowledge events

document.uploaded, version.ready, version.published, answer.cited

Consistency pattern

- Write case state and an outbox event in one Postgres transaction.
- A relay publishes committed outbox events to live UIs, webhooks and analytics consumers.
- Consumers record event IDs, making at-least-once delivery safe.
- Rebuild projections from events for recovery, audits and new user interfaces.

SECTION 09

Tool and CMR connector architecture

Keep the existing formulation catalog as the enforcement boundary and make backend substitution a tested adapter concern.

UI command -> workflow decision -> policy gate -> CHECK/WRITE formulation -> connector -> MockDB or real CMR -> typed result -> audit event

Layer	Responsibility	Contract test
Formulation catalog	Defines typed inputs, permission, approval, evidence, timeout, retries and result shape.	Catalog invariants: all writes require approval and idempotency; writes never blind-retry.
Policy gate	Checks plan size, allowlist, sensitivity and approval state.	Unknown tools and missing approvals are refused.
Tool client	Supplies identity, approval, evidence, case/run IDs and idempotency key.	Required context is present and correctly linked.
Connector interface	Translates canonical formulation to a backend call and canonical result envelope.	Same conformance suite against MockDB and real adapter.
Audit writer	Persists masked arguments, evidence, decision, latency and outcome before return.	Append-only, complete and privacy-safe.

Connector response envelope

```
{
  "schema": "cmr.tool-result/1.0",
  "status": "ok",
  "formulation_id": "CHECK.receipt.lookup",
  "data": {
    "status": "paid",
    "amount": 12500,
    "currency": "NGN"
  },
  "evidence_ref": "evidence_tool_8ac...",
  "audit_id": "audit_41d...",
  "error": null,
  "retryable": false
}
```

Pilot requirement: route normal task families into the CHECK/WRITE formulation plans so the user journey exercises the same boundary that a real CMR connector will use.

SECTION 10

Data, knowledge and search platform

Organize data by purpose: operational truth, immutable history, documents and query-optimized projections.

Store	Contains	Retention / access
Operational Postgres	Cases, inputs, extracted fields, tasks, approvals, runs, steps and current versions	Transactional; least privilege; point-in-time recovery
Event and audit tables	Case events, outbox records, tool audit and delivery receipts	Append-only; longer retention; restricted export
Object storage	Source PDFs, evidence images and generated exports	Encrypted; signed URLs; malware scan; lifecycle policy
Knowledge index	Published chunks, metadata, lexical/vector index and quality status	Only active approved versions answer users
Read projections	Case 360, work queues, SLA summaries and dashboard aggregates	Rebuildable; no unique business truth
Evaluation repository	Anonymized gold cases, expected outcomes and run results	Versioned with model, prompt, schema and code revision

Knowledge lifecycle

1. Ingest Parse document, preserve source hash and produce page-aware chunks.	2. Review Preview extraction, metadata, retrieval examples and sensitive content.
3. Publish Activate exactly one approved version per document; archive siblings.	4. Evaluate Run a gold-question suite for recall, citation relevance and groundedness.

Search evolution

- Keep the existing lexical scorer as a transparent baseline.
- Add hybrid retrieval: lexical plus embeddings, metadata filters and reranking.
- Return stable evidence references and exact quoted spans, not only document IDs.
- Measure retrieval recall separately from answer quality so failures are diagnosable.

SECTION 11

Security, privacy and governance

Make trust visible in the interface and enforce it again at every backend boundary.

Control	Architecture decision	User-facing expression
Identity	OIDC/SSO for staff; verified citizen session; service credentials for integrations	Named actor and organization shown on every decision
Authorization	RBAC plus case/region scope; tool permissions evaluated server-side	Disabled actions explain the missing permission
Approval	Separation of duties for defined writes and high-risk thresholds	Preview, evidence and approver identity before confirmation
Privacy	Field-level sensitivity, minimization, masking and export controls	Role-specific views hide unnecessary personal data
Integrity	Event hashes, optimistic versions, idempotency and append-only audits	Timeline distinguishes proposed, approved and executed
Transport/storage	TLS, managed secrets, database encryption and encrypted object storage	No secrets or raw tokens exposed in the UI
Retention	Documented periods by data class; deletion workflow with legal holds	Authorized staff can see expiry and hold status

Mandatory security gates before staff pilot

GATE

Reject missing credentials
When authentication is enabled, no protected endpoint may accept an absent credential.

GATE

Remove development secrets
Secrets must be injected from a managed store and rotated by environment.

GATE

Restrict browser origins
Allow only approved UI hosts; never use a production wildcard.

GATE

Exercise threat cases
Test injection, replay, stale updates, malformed JSON, oversized uploads and privilege escalation.

Safety metrics are gates, not averages: zero unauthorized writes, zero duplicate writes and 100% audited write attempts.

SECTION 12

Experience API and real-time contracts

Expose stable task-oriented resources. Keep commands explicit, queries fast and notifications compact.

Method and route	Purpose	Response
POST /v2/events	Generic signed event ingestion for trusted integrations	202 + event_id + processing status URL
POST /v2/cases	Create a case with initial JSON input	201 + case view
POST /v2/cases/{id}/commands	Submit add-message, assign, request-action or close commands	202 + command/event IDs
GET /v2/cases/{id}/view	Return the role-aware case workspace projection	200 + case_view/1.x
GET /v2/work-queues/{queue}	Return filters, counts and paged case cards	200 + queue_view/1.x
POST /v2/tasks/{id}/decisions	Approve or reject with reason and expected task version	202 + decision ID
GET /v2/schemas/{name}/{version}	Publish machine-readable JSON Schema	200 + schema
GET /v2/stream	Subscribe to permitted committed updates	SSE/WebSocket events

Notification payload

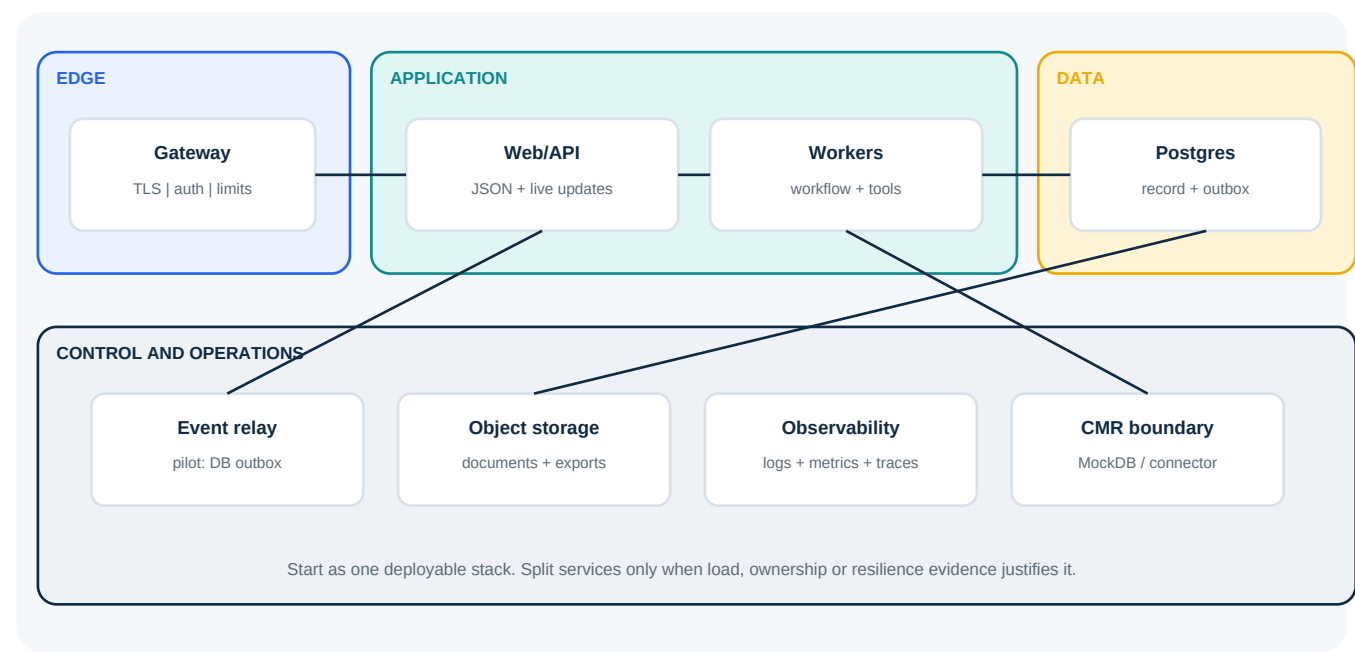
```
{
  "schema": "cmr.notification/1.0",
  "topic": "case.case_7f3.updated",
  "event_id": "evt_01k8...",
  "case_version": 14,
  "change": "task.waiting_approval",
  "refresh": [
    "case.summary",
    "case.actions",
    "case.timeline"
  ]
}
```

- Notifications contain references and view invalidation hints, not sensitive full records.
- Clients reconnect with a last-event cursor and recover missed updates through the query API.
- Every mutation accepts an idempotency key and expected resource version.
- OpenAPI documents HTTP contracts; JSON Schema documents domain payloads; AsyncAPI documents events.

SECTION 13

Pilot deployment and scaling path

Use a small number of independently scalable processes, but keep operational complexity proportional to the pilot.



Recommended pilot topology

Component	Pilot choice	Scale trigger
Web/API	Two stateless instances behind a gateway	Sustained CPU, connection or latency pressure
Worker	One worker group with controlled concurrency	Oldest queued task age breaches target
Database	Managed Postgres with backups and point-in-time recovery	Read replica or partition only after measured need
Event transport	Transactional outbox plus relay	External broker when fan-out, retention or throughput requires it
Documents	Encrypted object store	CDN only for approved public/static artifacts
Live updates	SSE for dashboards; WebSocket only for bidirectional needs	Connection scale beyond instance capacity

Do not begin with many microservices. The important architecture is the contract boundary: modules can be separated later without changing JSON schemas or user journeys.

SECTION 14

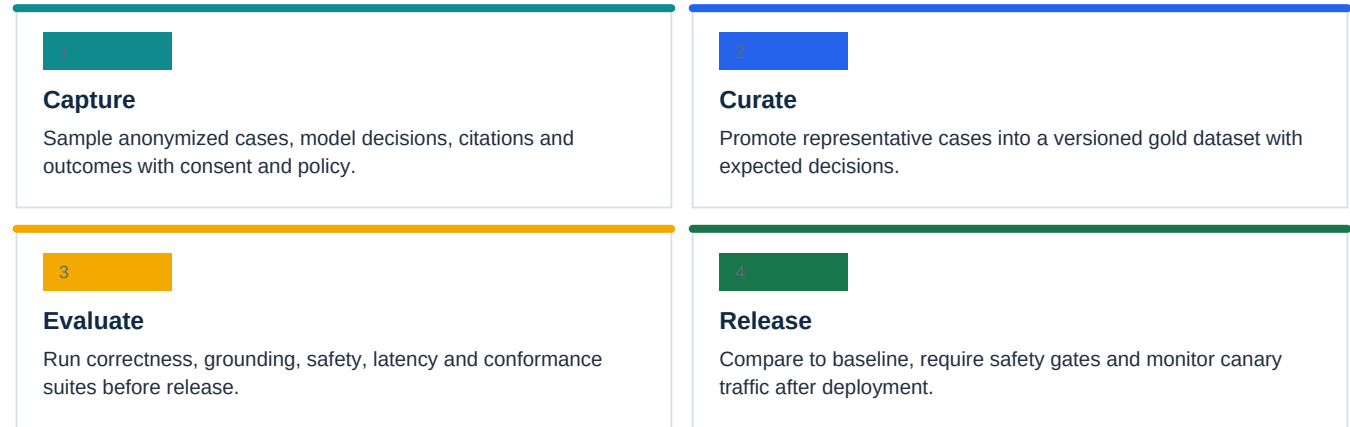
Observability and continuous evaluation

Every accepted event should be traceable across UI, API, workflow, tool call, backend result and audit record.

Operational signals

Signal	Measure	Pilot target
Intake health	accepted, rejected, quarantined and duplicate events by source	No unexplained loss; quarantine reasons visible
Queue health	oldest queued age, lease age, throughput and dead-letter rate	95% tasks start within agreed SLA
User experience	task completion, time-on-task, abandonment and correction rate	At least 90% scripted completion
Knowledge quality	retrieval recall, citation relevance, groundedness and fallback rate	Zero unsupported high-risk claims
Tool safety	writes by formulation, approval, evidence, duplicate and refusal reason	100% gated and audited
Reliability	API latency, error rate, worker restart recovery and webhook delivery	No duplicate side effects; scripted recovery passes

Continuous evaluation loop



Trace context

- Propagate request_id, event_id, case_id, task_id, run_id and audit_id through structured logs and results.
- Record code revision, schema version, workflow version, model/provider and prompt version on each run.
- Build a replay screen that reconstructs the exact inputs, decisions and outputs without exposing unnecessary PII.

SECTION 15

Implementation roadmap

Deliver user value in slices. Each phase ends with demonstrable workflows and explicit exit criteria.

Phase	Scope	Exit criteria
0. Contract alignment 1-2 weeks	Map CMR operations to formulation schemas; define event envelope; approve personas and core journeys.	Domain sign-off on MockDB realism and JSON v1 contracts.
1. Platform foundation 2-3 weeks	Mandatory auth, Postgres, transactional outbox, schema validation, idempotent POST and quarantine.	Continuous REST/webhook intake is replay-safe and observable.
2. Operator experience 3-4 weeks	Case 360, guided next action, evidence panel, approval inbox, timeline and live updates.	Operators complete payment and ownership scenarios without technical assistance.
3. Multi-UI expansion 3-4 weeks	Citizen PWA, supervisor dashboard, knowledge studio and integration console.	Role tests and accessibility checks pass.
4. Evaluation pilot 2-4 weeks	Gold dataset, adversarial testing, failure injection and shadow operations.	Hard safety gates pass; overall evaluation score at least 80%.
5. Real connector readiness	Implement real CMR adapter in a separate environment and run identical conformance tests.	No workflow/UI changes required; formal operational approval.

P0

Wire task families

Route payment, ownership, certificate, NIN and OTP cases into the existing CHECK/WRITE family plans.

P0

Publish JSON schemas

Add canonical event, command, case view, error and tool-result schemas with examples.

P0

Add outbox and projections

Emit committed events and build case_summary, queue_card and timeline read models.

P1

Build operator workbench

Replace the generic console with guided case manipulation and approval UX.

P1

Add live delivery

SSE channel with cursor recovery and role-filtered notifications.

P1

Automate evaluation

Run gold scenarios and connector conformance in CI; publish a release scorecard.

SECTION 16

Pilot definition and decision gates

A controlled pilot should prove the framework end to end while all CMR effects remain simulated.

Recommended pilot

Two to four weeks, staff-only, 50-100 representative cases, one controlled environment, synthetic or anonymized data, MockDB as the CMR simulator and human review of every proposed write.

Gate	Pass condition	Evidence
Functional	At least 95% valid-case workflow completion	Automated scenario report and sampled case replays
Safety	100% approval/evidence/idempotency controls; zero unauthorized or duplicate writes	Conformance, adversarial and audit completeness reports
Knowledge	No unsupported high-risk claims; citation relevance at least 90%	Domain reviewer rubric and retrieval report
Reliability	All scripted restarts and retries recover without lost accepted events	Failure-injection report
Usability	At least 90% scripted task completion and usability score at least 75/100	Observed sessions and questionnaire
Operations	Queue, errors, delivery failures and audit trails are visible to support staff	Dashboard and incident drill

Decision

This architecture is suitable for proving that CMR Specialist can accept continuous structured data, support intuitive role-based manipulation, enforce sensitive-action controls and preserve a complete audit trail. The decisive pilot milestone is not volume. It is demonstrating that every user action becomes a controlled, explainable and replay-safe JSON event from intake through simulated CMR outcome.

Deliverables for the architecture sprint

- Approved JSON schemas and examples for events, commands, views, errors and tool results.
- Clickable UI flows for the five experience surfaces, anchored on three priority journeys.
- Outbox, live-update and projection proof using the existing FastAPI, worker and database stack.
- End-to-end formulation routing against MockDB, with the conformance and evaluation scorecards attached.

Baseline repository components considered

harness/api, harness/agent, harness/store, harness/tools, harness/worker, harness/inference, frontend and the existing conformance tests. This proposal extends those boundaries; it does not replace the controlled-agent design.